

NykaaSight Conversational AI for Instant Business Intelligence Dashboards Hackathon Documentation

Prepared on: March 11, 2026

Prepared for: Hackathon Jury, Mentors, and Demo Audience

1) Executive Summary

NykaaSight is a full-stack web application that allows non-technical users to create interactive business dashboards using plain English.

Instead of writing SQL or manually configuring BI tools, a user can ask:
"Show me monthly revenue trend and best channels by ROI."

NykaaSight will:

1. Parse and understand the request using an LLM.
2. Retrieve relevant dataset context and prior chart memory.
3. Generate chart configurations in strict JSON format.
4. Process data and render interactive charts in real time.
5. Save chat history and dashboard snapshots for reuse.

This directly solves reporting bottlenecks for business teams and executives.

2) Problem Context (Simple Explanation)

Most business teams have lots of data but limited technical access.
If every question needs a data analyst, reporting becomes slow.

NykaaSight reduces this dependency by giving executives a conversational interface:

Ask in English -> Get dashboard instantly.

3) Business Problem Statement and Our Solution

Goal:

Build an intelligent system that converts natural language requests into complete, interactive BI dashboards.

NykaaSight solution:

- Accepts natural language queries.
- Chooses chart types based on context and chart rules.
- Aggregates and transforms data automatically.
- Renders chart outputs and textual insights.
- Supports follow-up requests ("filter this", "remove chart", etc.).
- Supports user CSV uploads (not hardcoded data only).

4) Target Persona

Primary persona: Non-Technical Executive (CX0)

What this persona needs:

- Fast answers.

- No SQL, no BI tool setup.
- Reliable numbers and clear visuals.

How NykaaSight helps:

- Natural language chat interface.
- KPI cards + dashboard charts.
- Snapshot saving for board meetings and review discussions.

5) Product Scope and Current Prototype Features

Frontend (React + Vite + Recharts):

- Upload page with drag-and-drop CSV.
- AI chat dashboard with interactive charts.
- KPI cards, chart controls, and suggested follow-ups.
- Snapshot save/load and print/export view.
- History page for datasets and sessions.
- Professional dashboard UI (custom hand-coded theme).

Backend (Node.js + Express):

- REST APIs for upload, chat, sessions, datasets, snapshots.
- CSV parsing, schema inference, chart data processing.
- LLM orchestration with NVIDIA NIM (OpenAI-compatible SDK).
- Vector memory integration for better contextual responses.

Storage (Supabase PostgreSQL):

- datasets
- chat_sessions
- chat_messages
- embeddings (vector memory)
- dashboard_snapshots

6) Architecture (End-to-End Flow)

Flow A: CSV Upload

1. User uploads CSV.
2. Server parses rows and infers schema.
3. Dataset metadata saved in Supabase.
4. Full rows cached in memory for fast chart processing.
5. Background vector indexing stores semantic chunks.

Flow B: Conversational Dashboard Generation

1. User asks a question.
2. Backend retrieves relevant vector context from embeddings.
3. LLM receives schema + sample rows + retrieved context + history.
4. LLM returns strict JSON (reply + chart configs + follow-ups).
5. Backend processes chart configs into chart-ready data.
6. Frontend renders charts using Recharts.
7. Chat and chart memory are persisted.

Flow C: Follow-up and Snapshot

1. User asks modification prompt ("only top 5", "remove pie chart").
2. Existing chart state is passed to LLM.
3. Updated chart configs are merged by chart id.
4. Snapshot can be saved and restored later.

7) Tech Stack Used (and Why)

Frontend:

- React (fast component architecture)
- Vite (fast dev/build experience)
- Recharts (practical interactive charting)
- Axios, React Router, Dropzone, PapaParse, Lucide icons

Backend:

- Node.js + Express (simple, reliable API layer)
- Multer + csv-parse (file upload and CSV parsing)

LLM:

- NVIDIA NIM API via OpenAI SDK compatibility
- Runtime model selection with fallback strategy

Database:

- Supabase PostgreSQL (managed storage + SQL + RPC)
- pgvector extension for semantic retrieval

8) API Surface (High-Level)

Core routes:

- POST /api/upload
- GET /api/datasets
- GET /api/datasets/nykaa/load
- GET /api/datasets/:id/preview
- POST /api/chat
- POST /api/chat/executive-dashboard
- GET /api/chat/kpis/:datasetId
- POST /api/sessions
- GET /api/sessions
- GET /api/sessions/:datasetId
- GET /api/sessions/:sessionId/messages
- POST /api/snapshots
- GET /api/snapshots
- GET /api/snapshots/:sessionId
- GET /api/snapshots/item/:snapshotId

9) Judging Rubric Mapping

9.1 Accuracy (40)

Data Retrieval:

- Chart data is computed from parsed dataset rows with filter, group, aggregate, sort, limit.
- Supports aggregation: sum, avg, count, max, min.

Contextual Chart Selection:

- Prompt instructions enforce chart type rules (time series -> line/area, part-of-whole -> pie, etc.).
- Backend now normalizes chart type variants for resilience.

Error Handling:

- Handles malformed AI JSON with parsing fallback and user-friendly error message.
- Handles missing dataset/cache and invalid upload cases.
- Degrades gracefully when vector memory retrieval fails.

9.2 Aesthetics and UX (30)

Design:

- Professional dashboard style with strong visual hierarchy.

Interactivity:

- Tooltips, legends, responsive charts, chart type switching, full-screen chart view.

User Flow:

- Guided upload states (uploading, indexing, done).
- Chat loading state and suggestion chips.
- Snapshot save/load and session history.

9.3 Approach and Innovation (30)

Architecture:

- Structured pipeline from text -> LLM -> processed chart data -> UI.

Prompt Engineering:

- Strict JSON-only response contract.
- Context injection with schema, sample rows, retrieved vector memory, and current dashboard state.

Hallucination Handling:

- System forces data-grounded outputs.
- When parsing fails or response is unclear, app returns safe user-facing message.

10) Bonus Criteria Coverage

Follow-up Questions (10):

Supported.

Users can chat with existing dashboard to refine filters, remove charts, or adjust focus.

Data Format Agnostic Upload (20):

Supported for CSV.

Users can upload their own CSV and start prompting immediately.

11) 10-Minute Demo Plan (3 Progressive Queries)

Demo setup (1 minute):

- Open app.
- Load "Nykaa Demo" dataset or upload custom CSV.

Query 1 (basic):

"Show monthly revenue trend and total conversions by month."

Expected: area/line + bar + quick insight text.

Query 2 (intermediate):

"Compare ROI by Channel_Used and show top campaign types by conversions."

Expected: ranking bars + channel comparison.

Query 3 (advanced follow-up):

"Now keep only top 5 channels, remove weak performers, and show segment-wise revenue share."

Expected: updated charts + pie/part-of-whole visualization + refined insights.

Wrap-up:

- Save snapshot.

- Open history.
- Export/print dashboard.

12) Submission Deliverables Checklist

1. Working prototype (Web App): Completed.
2. Public GitHub repository link: To be added by team.
Placeholder: <https://github.com/<your-org-or-user>/NykaaSight>
3. 10-minute presentation:
Use Section 11 demo script and Section 9 rubric mapping.

13) Deployment and Run Instructions

Environment:

- server/.env must include NVIDIA and Supabase keys.
- client/.env must include VITE_API_URL.

Run:

1. npm install
2. npm run dev

Default local URLs (current setup):

- Frontend: <http://127.0.0.1:5174>
- Backend: <http://localhost:5001>

14) Hackathon Pitch (Short Version)

"NykaaSight turns executive questions into live dashboards in seconds.

No SQL. No BI setup. Just ask and analyze.

It combines LLM reasoning, vector memory, and interactive visualization to make business intelligence truly conversational for non-technical teams."